

Evaluating the Impact of GPT-4o-mini-Based Identifier Renaming on Java Maintainability

Albaraa Alharbi, Reda Alamoudi, Abdulaziz Almalki, Mohammed Saifain

Department of Software Engineering

University of Jeddah

Jeddah, Saudi Arabia

Abstract—This study evaluates the effectiveness of GPT-4o-mini in improving Java code maintainability through semantic identifier renaming. A constrained instruction-based prompting strategy was applied to 20 Java source files containing low-quality or generic identifiers. The refactored code was evaluated using SonarQube metrics, including Code Smells, Maintainability Rating, and Cognitive Complexity. The results demonstrated a measurable reduction in code smells and an improvement in maintainability ratings while preserving the original program logic and cognitive complexity. These findings suggest that Large Language Models can support readability-focused refactoring workflows and contribute to sustainable software engineering practices.

Index Terms—Large Language Models, GPT-4o-mini, Semantic Renaming, Java Refactoring, Software Maintainability, Green Code.

I. INTRODUCTION

Beyond functional generation, Large Language Models (LLMs) improve software maintainability by enhancing code readability. This project uses Large Language Models (LLMs) to improve source code readability through semantic identifier renaming and context-aware naming improvements. The goal is to improve code readability, maintainability, and structural quality beyond the limits of traditional static analysis tools.

II. LITERATURE REVIEW (PART 1)

We reviewed ten key papers that define the current state of AI driven software construction:

- **Semantic Foundations:** *code2vec* [1] and *CodeT5* [2] demonstrate how neural models can learn the latent meaning of code structures for naming.
- **Practical Refactoring:** Studies on GPT-based repair [3] and readability improvements [4] show high human-acceptance rates for AI suggestions.
- **Comparative Performance:** Empirical investigations [5] highlight that while LLMs outperform novice programmers in refactoring, human expertise remains vital for complex tasks.
- **Optimization and Repair:** Advanced naming repair using models like Llama [6] and few-shot evaluation [7] underscore the importance of prompting strategies.
- **Human-AI Dynamics:** Research comparing developers with Large Language Models [8] and mining developer conversations [9] provides insight into how refactoring is articulated in real-world scenarios.

- **The Green Dimension:** Recent research [10] shows that AI-driven refactoring can reduce energy consumption by up to 82%, formalizing the concept of a "Green Code".

III. PROJECT PLAN (PART 2)

A. Research Questions

- **RQ1:** Can GPT-4o-mini improve the Maintainability Rating of legacy Java code through identifier optimization and semantic renaming techniques? [5].
- **RQ2:** Does semantic renaming lead to a measurable reduction in Cognitive Complexity compared to original code? [10].

B. Detailed Approach

We selected 20 Java source files containing low-quality generic identifiers (e.g., `var1`, `tmp_data`) from GitHub repositories and other sources. We applied a constrained instruction-based prompting approach using GPT-4o-mini to perform identifier optimization and semantic renaming where appropriate. We evaluate the improved code through readability comparison and static analysis tools such as SonarQube [5].

C. Evaluation and Presentation Plan

Success will be measured by improved Maintainability Ratings, reduction in Cognitive Complexity, and overall reduction of code smells as reported by static analysis tools such as SonarQube scores [5]. Results will be presented through Side-by-Side comparison tables and Bar charts highlighting changes in code quality.

IV. METHODOLOGY (PART 3)

This study follows a structured pipeline to evaluate the effectiveness of LLM-based identifier renaming in Java programs. The process begins with dataset preparation, where 20 Java source files containing low-quality identifiers were collected from academic sources and GitHub repositories.

A constrained instruction-based prompting strategy was applied using GPT-4o-mini through the OpenAI API. Each file was processed individually, ensuring that only identifiers were modified while preserving program logic and output.

The refactored files were then analyzed using SonarQube. The evaluation focused on Code Smells, Maintainability Rating, and Cognitive Complexity.

Pipeline Overview:

Input Code → Prompt Engineering → LLM Refactoring →
Clean Output → SonarQube Analysis → Metrics Comparison

A. Prompt Design

The following excerpt summarizes the main constraints and instructions used in the prompting strategy:

You are assisting in a software construction research project.

Task: Improve low-quality or generic identifiers in Java code.

Only rename:

- variables
- method names
- parameters

Strict constraints:

- Do NOT rename class names
- Do NOT change logic
- Do NOT modify algorithms
- Do NOT add or remove code

Output only Java code.

B. Dataset

The dataset consists of 20 Java files containing low-quality identifiers. These files were selected to simulate real-world maintainability issues.

The dataset is available as part of the replication package:
[GitHub Repository](#)

V. RESULTS

The results obtained from SonarQube indicate measurable improvements in code quality after applying LLM-based refactoring.

Code Smells: The number of code smells decreased across most files, with an average reduction of approximately 2.28 issues per file.

Cognitive Complexity: No significant change was observed in Cognitive Complexity, which was expected because the refactoring process intentionally preserved the original program logic and control flow.

This indicates that the improvements were primarily semantic and readability-oriented rather than structural or algorithmic.

Maintainability: Maintainability ratings improved from B to A after refactoring, indicating improved readability without structural modification.

Notably, the overall Maintainability Rating improved from B to A after applying LLM-based identifier optimization.

The results presented in Table I and Figure 1 demonstrate that LLM-based identifier renaming leads to a noticeable reduction in code smells while maintaining stable cognitive complexity.

TABLE I
SONARQUBE METRICS COMPARISON

Metric	Original	Refactored
Code Smells	5.14	2.86
Cognitive Complexity	3.00	3.00
Maintainability Score	B	A

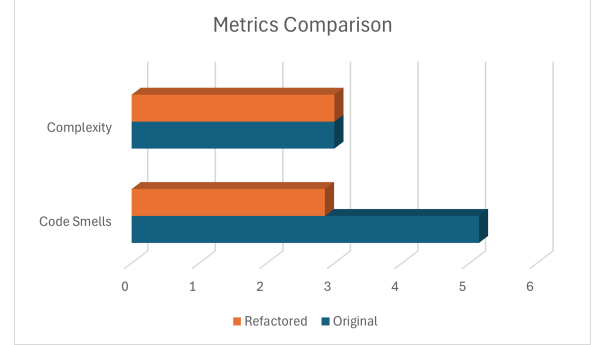


Fig. 1. Comparison of Code Quality Metrics Before and After Refactoring

This indicates that the improvements are primarily related to code readability and naming quality rather than structural or algorithmic changes. The reduction in code smells reflects enhanced clarity and better adherence to naming conventions.

However, the unchanged cognitive complexity confirms that the underlying program logic remains intact, which aligns with the constraints imposed in the prompting strategy.

VI. DISCUSSION

Although the results suggest improvements in code quality, manual inspection revealed limitations.

In some cases, the model introduced inconsistencies across related classes, where identifiers were renamed in one class but not updated in dependent classes.

This indicates that while LLMs improve readability, they may fail in maintaining semantic consistency.

Additionally, static analysis tools such as SonarQube were unable to detect these inconsistencies, highlighting their limitations in evaluating logical correctness.

VII. CONCLUSION

This study explored the effectiveness of GPT-4o-mini in improving Java identifier naming to support software maintainability and Green Code practices.

The results demonstrated that LLM-based refactoring can significantly reduce code smells and improve readability while preserving the original program structure and logic.

However, manual inspection revealed that semantic inconsistencies may still occur across related classes, highlighting the importance of combining automated refactoring with human validation.

Overall, the study confirms that Large Language Models can serve as valuable assistants in software construction workflows, particularly for readability-focused refactoring tasks.

VIII. REPLICATION PACKAGE

To ensure reproducibility, the dataset, scripts, prompts, and results are provided in the following replication package:

[GitHub Repository](#)

REFERENCES

- [1] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, “code2vec: Learning distributed representations of code,” *Proceedings of the ACM on Programming Languages*, 2019.
- [2] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *EMNLP*, 2021.
- [3] R. Ishizue, K. Sakamoto, H. Washizaki, and Y. Fukazawa, “Improved program repair methods using refactoring with gpt models,” in *SIGCSE*, 2024.
- [4] A. Vitale, V. Piantadosi, S. Scalabrino, and R. Oliveto, “Using deep learning to automatically improve code readability,” in *ASE*, 2023.
- [5] G. Saba, S. Ahmed, H. Sania, T. A. Khan, and H. B. M. Nasir, “An empirical comparison of ai assisted software refactoring tools,” *Scientific Reports*, 2026.
- [6] M. Yousuf, A. Bagade, M. Baraya, and C. Penugonda, “Neural variable name repair: Learning to rename identifiers for readability,” *arXiv preprint arXiv:2512.01141*, 2025.
- [7] M. R. Tapader, M. M. Rahman, A. I. Shiplu, M. F. I. Amin, and Y. Watanobe, “Code refactoring with llm: A comprehensive evaluation with few-shot settings,” *arXiv preprint arXiv:2511.21788*, 2025.
- [8] N. Nascimento, P. Alencar, and D. Cowan, “Comparing software developers with chatgpt: An empirical investigation,” *arXiv preprint arXiv:2305.11837*, 2023.
- [9] E. A. AlOmar, A. Venkatakrishnan, M. W. Mkaouer, C. D. Newman, and A. Ouni, “How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations,” in *MSR*, 2024.
- [10] S. Rashed, “Llms to automatically refactor green code anti-patterns,” Master’s thesis, University of Amsterdam, 2025.